

Large (second) assignment

Common requirements for the assignments

You need to create documentation (a pdf file) for the assignment. The program is only acceptable if you also have the documentation.

There are some things that are not defined, for example, exact numbers for durability loss, fuel consumption, probability thresholds, etc. In these cases you are free to design and implement them as you see fit.

The documentation should include

- the description of the exercise (copied from here)
- a use case diagram
- a class diagram (designed using Umbrello, draw.io, PlantUML, etc.)
- a state diagram (for the specific entity described in each assignment)
- two sequence diagrams (for the scenarios described in each assignment)

The code should include

- javadoc comments for each class and method (except getters and setters)
- unit tests for each of the important methods (not getters or setters)

1 University Library Management System

The university library management system maintains a diverse collection of items and facilitates borrowing and returning processes for members. Members interact through a unified interface, but behind the scenes the process unfolds differently depending on the item type.

1.1 Items

Items have an id, a title, an author, page count, and topic. The system tracks how many times an item has been borrowed. An item is one of the following concrete types:

1.1.1 Books

Books are the most common item for borrowing. A book may have multiple physical copies, and new copies can be added at any time. When a book is requested, the system checks for available physical copies. If at least one copy exists, the book is lent to the member and both the item and member records are updated. If all copies are out, the book is marked as unavailable for others until a return occurs. A book can also have a digital copy (see Digital Copies below).

1.1.2 Theses

Theses are handled with similar rules to books: they may be borrowed and returned, and availability depends on physical copies. However, they are typically rarer, often with just one physical copy. No digital copies are produced for theses. In addition to the common item data, a thesis stores an advisor and a defense year; this information is retained for academic referencing but does not influence borrowing logic.

1.1.3 Online Research Papers

Online research papers are not physically borrowed. When a member requests one (i.e. tries to borrow), the system prepares a digital access instance instead. The paper is immediately made available regardless of how many others are viewing it. Each access is recorded as a view, and if the member chooses to download it, the system logs the download and increments the usage count.

1.2 Digital Copies

An item can have a digital copy (books only; theses and online research papers follow their own rules described above). By default an item would not have one, but it can be created. When created, it has the same id as the item. Members can view the item and, if they want, download the digital copy. The number of views and downloads is kept. The library should be able to find the most viewed and most downloaded items.

1.3 Item Lifecycle (State Diagram)

The state diagram for a Books or Thesis Item captures the dynamic transitions between various states. Initially an item is in the **Available** state, indicating that it can be borrowed. When a member borrows it the item transitions to **Borrowed**, but only if there are available physical copies. If all copies are borrowed it moves to **Unavailable**. Upon return, if the item is marked as **Preserved** it returns to the Available state; however, if returned as **Damaged** it transitions to a **Needs Repair** state. From there the item either proceeds to the **Being Repaired** state or, if deemed severely damaged (based on a probabilistic condition), transitions to **Archived**, permanently removing it from circulation.

1.4 Members

To borrow items, users must be members of the library. A member has an id, name, email, and a category (student, faculty, or alumni). Each member keeps an active list of currently borrowed items; initially this list must not exceed 3. Members also keep a count of total successfully returned items. For every 3 items that are returned in preserved condition (not damaged), the member's loyalty is appreciated by increasing their allowed borrowing limit by one.

1.5 Repairs Section

The library has a Repairs Section. It keeps count each time an item is sent to be repaired. If damage is severe (use randomness) then the item is removed from the library's active collection and moved to the archives. If successfully repaired the item normally becomes available again. The library does a periodical check every now and then to send items that need repair to the Repairs Section and to execute the repair process.

1.6 The Library

The library stores the collection of items. It also tracks active transactions (borrowing and returning) and provides search functionalities by title and author to help users locate desired materials. The library can add new items to its collection and manages the borrowing and returning of items. It also keeps track of all its members and can search for the most active member. Users can find the most popular items based on the number of times they have been borrowed: the most popular digital item and the most popular item in general, taking into account both physical and digital usage.

1.7 Simulation

The main simulation must initialise the library, create items and add them to the library (some items might have digital copies), create members, and make borrowing and returning transactions as well as other necessary operations such as searching for specific items or making repairs. Read the input from files; use as many input files as needed.

1.8 Scenarios to model with sequence diagrams

1.8.1 Scenario 1: Book Borrowing with Condition Tracking

A student borrows a book with multiple copies in circulation. The system verifies availability, assigns a copy to the student, and records the transaction. Upon return the book is inspected and marked as preserved, incrementing the student's loyalty progress. The book becomes available again.

1.8.2 Scenario 2: Damaged Return and Removal

A faculty member returns a thesis copy in damaged condition. The system flags the item and sends it to the repair process. After evaluation the damage is deemed irreversible. The thesis is removed from borrowing circulation and marked as archived.

Input file examples

items.txt

- Format

ID	Title	Author	Page_Count	Topic	Type	Copies
----	-------	--------	------------	-------	------	--------

- Sample input

```
1 Clean_Code Robert_C_Martin 464 Software_Engineering Book 3
2 The_Pragmatic_Programmer Andrew_Hunt 352 Software_Engineering Book 2
3 Design_Patterns Erich_Gamma 395 Software_Engineering Book 2
4 Refactoring Martin_Fowler 448 Software_Engineering Book 3
5 AI_Thesis John_Doe 120 Artificial_Intelligence Thesis 1
6 Deep_Learning_Survey Jane_Roe 30 Machine_Learning ResearchPaper 0
```

members.txt

- Format

```
ID Name Email Category
```

- Sample input

```
1 Bence_Nagy bence@elte.hu Student
2 Eszter_Kovacs eszter@elte.hu Faculty
3 John_Smith john@elte.hu Alumni
4 Levente_Toht levente@elte.hu Student
5 Sophia_Martinez sophia@elte.hu Faculty
```

2 Recipe Book System for a Restaurant

The Recipe Book System is designed to provide restaurants with a structured and extensible way to manage their recipes and their inventory to deliver the best and most fresh food.

The recipe book holds a list of all available recipes. New recipes can be added and removed. The system provides users with the ability to search and filter recipes by name, cuisine, or ingredient, offering a highly customizable browsing experience.

Each recipe has a name, cuisine, list of ingredients, and cooking time. For each recipe you can add and remove ingredients and get recipe details. We are not modelling quantities of the ingredients; each recipe requires exactly one of each. The total cooking time for a recipe can be calculated from the preparation times of all its ingredients.

2.1 Ingredients

Each ingredient has a name, category, and unit of measurement. Ingredients are classified into categories such as dairy, meat, or vegetables, making it easier for chefs to organise and filter their recipes. The system also tracks how many recipes use each ingredient.

To better reflect real kitchen workflows, there are three distinct ingredient types, each of which calculates preparation time differently based on the specific steps required to ready it:

2.1.1 Meat

Meat typically requires thawing if frozen, followed by cooking, making it one of the more time-consuming ingredients to prepare. Total preparation time is the sum of thawing time and cooking time.

2.1.2 Vegetables

Vegetables are faster to handle but still require cleaning and chopping. The system adds washing time and cutting time to determine readiness, with variations depending on the type and freshness of the vegetable.

2.1.3 Bakery Items

Bakery items such as doughs and batters involve a different process — they need time to rest or rise before baking. Total preparation time factors in both resting and baking durations.

2.2 Recipe Lifecycle (State Diagram)

A recipe begins in an **Idle** state, waiting in the system until it is requested by a chef. Once selected, the system evaluates whether all necessary ingredients are available in stock. If everything is ready, the recipe proceeds directly to **Preparation**. If even one required item is missing, the system halts the process, flags the missing ingredients, adds them to the shopping list, and places the recipe **On Hold** until restocking is complete. Once all components are present, the kitchen begins preparation. After cooking is completed, the system finalises the recipe, updates ingredient usage statistics, and resets the recipe's readiness, returning it to **Idle** so it can be selected again.

2.3 Restaurant

The restaurant keeps track of its recipes through a recipe book, its ingredients in stock, and a shopping list. To prepare a recipe, the restaurant must check if all ingredients are available in stock; if not, the restaurant cannot prepare the recipe and the missing ingredient is added to the shopping list to be bought later. If all ingredients are available, the recipe is prepared and the ingredients are taken from stock. The recipe book can search for the most used ingredients and the most popular ingredient categories.

2.3.1 Ingredient Stock

For each ingredient the stock tracks its quantity. Stock can be added whenever needed, and the system can check whether an ingredient is available. Stock is reduced whenever a recipe is prepared.

2.3.2 Shopping List

The shopping list contains ingredients that must be bought. Ingredients are added when their stock runs out. When buying, either the whole shopping list is purchased or only a single item. When an item is bought it is removed from the shopping list and added back to stock.

2.4 Simulation

Simulate the process by initialising a restaurant object and adding a recipe book to it. The restaurant also has shopping and stock lists. Chefs should be able to cook recipes by using ingredients available in stock, and when needed, items on the shopping list should be bought. Read the input from files; use as many input files as needed.

2.5 Scenarios to model with sequence diagrams

2.5.1 Scenario 1: Preparing a Recipe with Complete Stock

The chef chooses a pasta recipe. The system confirms that all ingredients — including pasta, tomatoes, olive oil, and cheese — are available in the required quantities. The dish is prepared, and each ingredient's quantity in stock is reduced accordingly. The ingredient usage count is updated, and the preparation is logged.

2.5.2 Scenario 2: Missing Ingredient Handling

A dessert recipe is requested. However, the system detects that eggs are not available in stock. The preparation is halted, and “eggs” are added to the shopping list. The system marks the recipe as pending until the missing ingredient is purchased.

Input file examples

ingredients.txt

- Format

Name Category Unit_Of_Measurement

- Sample input

Spaghetti Pasta Grams
Ground_Beef Meat Kilograms
Tomatoes Vegetables Kilograms
Bread_Dough Bakery Grams

recipes.txt

- Format

Name Cuisine Ingredients Cooking_Time

- Sample input

Spaghetti_Bolognese Italian Spaghetti-Ground_Beef-Tomatoes-Garlic-Olive_Oil 30
Chicken_Alfredo Italian Chicken-Alfredo_Sauce-Garlic-Parmesan-Fettuccine 25
Veggie_Burger American Veggie_Patty-Lettuce-Tomato-Onion-Bun 20

3 Venue Booking System in University

The university provides different types of venues that can be booked by anyone with a university email, including staff and students. The booking of all venues is done through one system per university. Each venue belongs to a specific department of the university.

3.1 Departments

Venues are managed by departments (for example: Informatics, Languages, etc.). A department has a physical location. Each department also has a contact person whose information is available when help is needed making reservations. The department has a list of available equipment that can be rented when a venue reservation is made. A department can have multiple venues or no venues, but a venue must belong to a department. Departments track rental usage, including frequently requested items and revenue collected.

3.2 Venues

The university offers three types of venues tailored to different needs:

3.2.1 Small Team Rooms

Team rooms are compact, seating up to five people, and can be booked instantly for short slots. They may include a whiteboard or a smart board. Bookings are free. A routine maintenance check is performed automatically after every 10 uses.

3.2.2 Classroom-Sized Halls

Classroom halls vary in size and always include a whiteboard and a digital device; they may also include other equipment. The booking fee is based on a base amount (fixed for all classrooms) plus the number of equipment items in the room. A routine maintenance check is performed after every 5 events.

3.2.3 Big Event Halls

Big event halls support large-scale activities and may include features like podiums, sound systems, stages, and other equipment. The booking fee is calculated based on both the number of seats and the available equipment. A maintenance check is performed after every event.

For all venue types, if a venue requires additional repairs beyond standard maintenance, a maintenance request is generated. Users should be able to get details of a venue. Double booking must not be allowed.

3.3 Venue Lifecycle (State Diagram)

Each venue transitions through a set of operational states depending on its usage and condition. Initially a venue is **Available** and ready to be booked. Once

a reservation is confirmed it moves into a **Booked** state, and when the scheduled event begins the venue is marked as **In Use**. After the required number of uses (depending on venue type) it is flagged for a standard maintenance check, placing it in a **Pending Maintenance** state. If the inspection reveals any issues the venue enters an **Under Repair** phase, during which it becomes temporarily unavailable for booking. Once maintenance or repairs are completed the venue becomes **Available** again, restarting the cycle.

3.4 Equipment

Each piece of equipment has a name (projector, microphone, etc.), a type (audio, visual, furniture), a brand, and an extra price (can be zero). Some venues come with permanently installed equipment that cannot be rented separately. However, departments offer additional rentable equipment that users can request when booking a venue.

3.5 Users

Users must be university members verified by their email. The user type is one of these: admin, student, or faculty. Users can make, cancel, and view reservations, with a total count of their bookings tracked.

3.6 Booking System

The university has a single centralised booking system that manages all venue reservations. It has a list of all available venues at the university and provides the following functions:

- Browse available venues and check real-time availability.
- View venue details, including installed equipment and rental options.
- Make, modify, or cancel bookings based on user role. Request additional equipment from the venue's managing department.
- Track maintenance schedules and report venue issues.

3.7 Simulation

The main simulation must initialise the university and its departments. For each department add the venues and the equipment. Add users so that they can make bookings of venues and request extra equipment using the booking system. Read the input from files; use as many input files as needed.

3.8 Scenarios to model with sequence diagrams

3.8.1 Scenario 1: Booking a Conference Room

A staff member browses available venues. The system filters and shows conference rooms. The user selects a room and requests additional equipment (e.g., a microphone). The system checks room availability and equipment stock. Booking is confirmed, the room's booking count is incremented, and rental revenue is updated for the department.

3.8.2 Scenario 2: Booking Fails for a Big Event Hall

A university club member requests a Big Event Hall for an upcoming show. The system checks the venue's availability. The system finds the venue is not available. Booking is denied and the user is notified.

Input file examples

users.txt

- Format

Name Email Type

- Sample input

```
Sandor sandor@elte.hu admin
Peter peter@elte.hu admin
Anna anna@bme.hu faculty
Zoltan zoltan@corvinus.hu student
David david@elte.hu faculty
Eva eva@bme.hu student
```

departments.txt

- Format

Name Location ContactPerson

- Sample input

```
IK Lágymányos Sandor
TTK Lágymányos Peter
```

venues.txt

- Format

Name Capacity Department Type Equipment

- Sample input

Main_Lecture_Hall 200 IK BigEventHall Projector-Microphone-Whiteboard
Small_Meeting_Room 5 IK TeamRoom Whiteboard
Physics_Lab 50 TTK Classroom Lab_Benches-Fume_Hood
Business_Conference_Room 40 IK Classroom Smart_Board-Video_Conference_System
Workshop_Hall 80 TTK BigEventHall Work_Benches-3D_Printer

equipment.txt

- Format

Name Type Brand RentalPrice Department

- Sample input

Projector Visual Epson 5000 IK
Microphone Audio Shure 2000 IK
Whiteboard Furniture Nobo 0 IK
Conference_Phone Audio Polycom 3000 IK
TV_Screen Visual LG 4000 TTK
Smart_Board Visual Samsung 7000 TTK
Fume_Hood Furniture Thermo_Fisher 0 TTK
3D_Printer Visual Prusa 15000 TTK

4 Orion Expedition

In the vast reaches of space, a powerful space shuttle embarks on a voyage to explore distant planets. To navigate this journey, the shuttle relies on several key systems that work together to reach its destinations efficiently.

4.1 Destinations

Destinations are planets or celestial bodies the shuttle aims to reach. Each destination has a name, a distance from Earth, a scientific value metric, an observation completion percentage, a specific fuel consumption rate, and a travel progress tracker. The destination has a formula to calculate how far the shuttle is from it so that it can update its progress. All planets are assumed to be in a line, so the distance is simply “distance of destination from Earth” minus “how far we have come from Earth”.

4.2 Navigation System

The navigation system acts as the shuttle’s guide, maintaining a list of destinations. It allows adding new destinations and removing ones that have been fully reached. The navigation system features three distinct routing strategies:

- **Standard mode** takes the direct approach, determining which destination is nearest to the shuttle.
- **Efficient mode** factors fuel consumption into routing decisions by calculating a fuel-to-distance ratio for each destination.
- **Exploration mode** chooses the destination with the highest scientific value, sometimes picking longer paths deliberately to study cosmic phenomena of interest.

4.3 Fuel Tank

The tank is responsible for fuel management: it keeps track of the current fuel and the maximum fuel capacity. The tank reduces the available fuel based on a percent usage, while refuelling restores the tank to full capacity. If the fuel drops below 15% of its maximum level, the system signals that it is low on fuel.

4.4 Main Engine

The main engine provides the thrust needed for space travel. It can start and stop as required. Moving in the cruising state consumes fuel by 10% and increases the shuttle’s progress toward its next destination by 5%.

4.5 Orbiter

The orbiter is the shuttle’s navigator. It has a current destination (starting with Earth), a next destination, and a reference to the shuttle. To start the course the

orbiter takes the next destination, starts the engine, and initiates movement while the engine is running. If fuel runs low it attempts to refuel at a 50% chance before continuing; otherwise the engine stops running. If the shuttle reaches its target the orbiter updates its current destination.

4.6 Shuttle Lifecycle (State Diagram)

The shuttle's operational behaviour is governed by a state model. Initially the shuttle is in the **Docked** state where it can refuel but cannot move as the engine remains inactive. When the engine starts it transitions to the **Cruising** state, travelling at standard velocity by consuming fuel per move and increasing travel progress by 5%. Upon reaching a destination the shuttle enters the **Orbiting** state, performing observations with reduced fuel consumption at 3%. If fuel levels drop critically low the shuttle transitions to an **Emergency** state where it stops observation and attempts to refuel. If refuelling succeeds the shuttle returns to Cruising; if unsuccessful the engine stops and the shuttle returns to Docked.

4.7 Space Shuttle

At the heart of it all is the space shuttle, which ties these components together. It has a navigation system, an orbiter, an engine, and a fuel tank. The shuttle can add destinations and launch into space, driving the orbiter towards the closest destination (or the best one according to the active navigation strategy). When a destination is successfully reached it is removed from the navigation. The launch is complete when all destinations have been reached.

4.8 Simulation

The main simulation serves as the entry point, initialising the space shuttle, setting destinations like Mercury, Jupiter, Saturn, Neptune, and Mars, and launching the expedition. The shuttle systematically travels from one destination to another, overcoming fuel challenges along the way. Once all planets are visited, the mission is deemed a success. Simulate, launch the shuttle, and show the actions!

4.9 Scenarios to model with sequence diagrams

4.9.1 Scenario 1: Successful Planetary Visit

The navigation course begins with the selected destination. The engine starts and the shuttle enters the Cruising state. As the shuttle advances, its movement gradually increases the destination's travel progress until it reaches 100%. Once

complete, the current destination is updated, the destination is removed from the navigation system, and the closest destination becomes the next target using the active navigation strategy.

4.9.2 Scenario 2: Fuel Emergency Management

When fuel drops below 15% of capacity the shuttle enters the Emergency state. The system attempts refuelling with a 50% success chance. If successful, the fuel is restored to maximum and the mission continues. If unsuccessful the engine stops, halting progress until the situation is resolved. The shuttle can implement an enhanced emergency protocol that enables solar energy collection in deep space, modifying the random refuelling chance based on proximity to stars, or calculating a minimum-energy return trajectory to the nearest known refuelling point by selecting the closest destination with refuelling capabilities.

Input file examples

destinations.txt

- Format

Name Distance ScientificValue FuelRate

- Sample input

```
Jupiter 1000 8 12
Saturn 500 7 10
Neptune 200 9 15
Mars 4000 6 8
```

5 Hospital Emergency

In a bustling city, emergencies strike at unexpected moments. When a medical crisis occurs, the city's emergency alert system ensures that patients receive the care they need as quickly as possible. The system relies on ambulances, hospitals, and doctors working together to save lives.

5.1 Patients

Patients are individuals in need of medical attention. Each patient has a name, a severity level ranging from 1 (mild) to 10 (critical), and a status indicating whether

they have fully recovered. When a doctor treats a patient, their severity level is reduced based on the doctor's efficiency. If the severity reaches zero the patient is marked as treated.

5.2 Patient Lifecycle (State Diagram)

The emergency system tracks patients through a state model. Initially patients enter the **Waiting** state, where they undergo initial assessment with severity assigned but remain not transported. When an ambulance is dispatched, patients enter the **Transit** state as they are transported to a hospital. Upon arrival patients transition to the **Admitted** state, being added to the hospital's patient list. When a doctor becomes available patients move to **Under Treatment**, receiving care with severity gradually reduced. Finally, when severity reaches zero patients transition to a **Recovered** state, marked as treated and ready to discharge.

5.3 Ambulances

Ambulances transport patients. Each ambulance has an identifier, a status indicating if it can transport a patient, and a destination hospital where it will take patients.

5.4 Hospitals

Hospitals are the core of the system, where treatment takes place. A hospital has a name, a capacity defining the number of patients it can accommodate, and a list of both patients and doctors. The hospital registers a patient if there is space and can also register a doctor to its staff. When it is time for treatment the hospital instructs all doctors to care for all untreated patients using one of the following treatment strategies:

- **Standard treatment** applies doctors' efficiency directly to reduce patient severity following established protocols.
- **Intensive treatment** adds a $1.5\times$ efficiency multiplier for patients with severity above 5, using more resources for faster healing.
- **Emergency treatment** sorts patients by urgency using a formula ($\text{severity} / 10 + 1$) to boost efficiency for critical cases.

5.5 Doctors

Doctors are responsible for treating patients. A doctor has a name and an efficiency level, which determines how much the doctor can reduce a patient's severity per session.

5.6 Emergency Alert System

Overseeing all operations is the emergency alert system. It manages all hospitals and ambulances, ensuring that patients are taken to available hospitals. The system allows new hospitals and ambulances to join and retrieves a hospital not at full capacity, if any. When a patient needs transport, the system finds an available hospital and the first available ambulance. If no hospitals have space or no ambulances are free the patient must wait. Once patients arrive at hospitals the system ensures that doctors begin their work.

5.7 Simulation

The main simulation ties everything together. It initialises the alert system, registers hospitals and doctors, assigns ambulances, and simulates patient arrivals. Simulate, start the treatments, and show the actions!

5.8 Scenarios to model with sequence diagrams

5.8.1 Scenario 1: Routine Emergency Response

When a new patient arrives with assessed name and severity, the emergency alert system identifies the best facility and dispatches an ambulance. The ambulance is assigned to the hospital and transports the patient. Upon arrival the hospital registers the patient, then begins treatment. Doctors apply their efficiency to reduce patient severity until the patient is fully treated and can be discharged.

5.8.2 Scenario 2: System Under Capacity Constraints

During mass casualty events with many patients the system's capacity is tested as hospitals approach their patient limits. As the system struggles to find hospitals with available capacity, incoming patients must wait. The system implements priority scheduling by sorting patients by severity before treatment, ensuring doctors treat critical patients first. If all hospitals reach capacity the system activates additional resources by temporarily increasing hospital capacity by 25% for emergency overflow and implements accelerated treatment protocols. Throughout this

process each patient's state transitions are carefully tracked to ensure no cases are overlooked.

Input file examples

hospitals.txt

- Format

Name Capacity

- Sample input

General_Hospital 3
City_Medical_Center 2

doctors.txt

- Format

Name Efficiency Hospital

- Sample input

Dr_Smith 5 General_Hospital
Dr_Adam 3 City_Medical_Center
Dr_Farizi 8 General_Hospital

ambulances.txt

- Format

ID

- Sample input

A1
A2
A3
A4

patients.txt

- Format

Name Severity_Level

- Sample input

John 7
Alice 4
Bob 9
Eve 6
Charlie 3

6 Adventurer's Guild

In a world of adventure, a guild recruits adventurers to take on dangerous quests. Each adventurer arms themselves with weapons and faces formidable monsters to prove their strength. The guild also offers a repair service to restore weapons' durability.

6.1 Weapons

Weapons represent equipment that boost an adventurer's power. A weapon has a power boost, which increases attack strength, and a durability, which determines how many times the weapon can be used. The weapon can decrease its durability, and checks if it is unusable. The weapon can also return the power increase, if it is still functional (see weapon states below).

The guild offers three specialised weapon types:

6.1.1 Swords

Swords excel in melee combat, delivering powerful strikes with a 20% chance of inflicting critical hits that double damage output — though critical hits cost 2× durability points.

6.1.2 Bows

Bows specialise in ranged combat, providing precision attacks that bypass 30% of the enemy's armour. Durability reduction is proportional to the target's armour.

6.1.3 Stuffs

Stuffs harness magical energy, launching spells with a 25% chance to halve a monster's strength. Durability consumption is proportional to the staff's strength.

6.2 Weapon Lifecycle (State Diagram)

The equipment system tracks weapons through four distinct states that affect combat performance. In the **New** state (100% durability), weapons provide their full power bonus with a 10% first-use bonus damage effect. After first use weapons transition to the **Used** state (50–99% durability), maintaining standard performance with their regular power boost. With continued use weapons enter a **Damaged** state (1–49% durability), with their power boost reduced to 75%. When durability reaches zero weapons transition to the **Broken** state, becoming unusable. Before an adventurer's next quest, they attempt to repair their weapons by spending experience points proportional to each weapon's power — if successful, the weapon returns to the New state.

6.3 Monsters

Monsters are the creatures that adventurers battle. They have a name, strength, health, and armour. A monster reduces its health based on incoming damage and its armour, and can check if its health reaches zero (then it dies). The monster can also return its attack power and defence.

6.4 Adventurers

An adventurer is a hero with a name, base power, experience, rank, health, and an inventory of weapons. They can equip weapons which boost their attack power. The damage of the adventurer's attack is calculated as: $\text{base power} \times (10\% \text{ of rank}) + \text{total strength of weapons (adjusted by weapon state)}$. The adventurer's health can be decreased, and they can fall in battle if it reaches zero. After each battle the adventurer fully restores their health. Adventurers use their weapons in combat, gain experience, and increment their rank and power up by 20% when enough experience is gained — which is when their experience is greater than or equal to ten times their rank. The adventurer can also initiate a quest for themselves.

6.5 Quests

A quest represents a mission with a name, difficulty, a status to check if it's completed, and a list of monsters. The quest is done by an adventurer who fights

each monster sequentially. If an adventurer succeeds the quest is marked complete; otherwise it is incomplete. The adventurer takes turns to combat each monster, alternating attacks until one is defeated. When the adventurer attacks they use their strongest non-broken weapon first (losing durability), then deal damage (power - monster's armour); if the monster is not defeated they receive damage from the monster. A defeated monster lets the adventurer gain experience.

6.6 Guilds

Guilds manage adventurers and quests. A guild has a list of adventurers and a quest board. The guild can add a new member and introduce new quests. The guild assigns a random adventurer to quests, letting them attempt challenges sequentially until all quests are completed.

6.7 Simulation

The main game runs the simulation. It establishes a guild, recruits the adventurers, equips them with weapons, and posts quests. Simulate, start the quests, and show the actions!

6.8 Scenarios to model with sequence diagrams

6.8.1 Scenario 1: Successful Quest Completion

When an adventurer attempts a quest they first equip appropriate weapons. During monster fights the adventurer uses their strongest non-broken weapon, dealing damage based on calculated power against monster armour. As the adventurer defeats monsters they gain experience that increases rank and base power by 20% when experience reaches threshold values. After defeating all monsters in the quest's list, the quest is marked as complete and the adventurer recovers full health.

6.8.2 Scenario 2: Equipment Failure

During a challenging quest against a high-health monster, weapons sustain damage with each use, decreasing durability. As durability falls below 50% the weapon enters the Damaged state, reducing power by 25%. If durability reaches zero during combat the weapon provides no power boost. This sudden power reduction may leave the adventurer unable to overcome the monster's armour value, causing them to take damage without dealing any in return. If defeated, the quest attempt fails and the adventurer must recover before attempting another quest.

Input file examples

adventurers.txt

- Format

Name Health Base_Power

- Sample input

Aragorn 50 30
Legolas 40 20
Gimli 60 40

weapons.txt

- Format

Name Type Power Durability Owner

- Sample input

Anduril Sword 35 100 Aragorn
Elven_Bow Bow 25 80 Legolas
Dwarven_Axe Sword 40 90 Gimli
Gandalf_Staff Staff 30 70 Legolas

quests.txt

- Format

Name Difficulty Monsters

- Sample input

Defeat_the_Goblin_King 70 Goblin_Warrior-Goblin_Warrior-Ogre-Goblin_King
Rescue_the_Village 50 Ogre-Ogre-Bandit_Leader
Slay_the_Dragon 100 Fire_Dragon

monsters.txt

- Format

Name Health Attack Armour

- Sample input

```
Goblin_King 70 100 15
Goblin_Warrior 40 50 5
Ogre 50 80 10
Bandit_Leader 60 70 12
Fire_Dragon 1000 200 50
```

7 Borderland Survival

In a distorted other world known as the Borderland, some creatures are forced to play deadly games to survive. Every game tends to test intellect, brawn, teamwork, or brutality. Players must scavenge for supplies, form alliances, and solve increasingly complicated puzzles in their efforts to remain alive (forming alliances and scavenging are not modelled).

7.1 Players

Every player has a name, a mental stat, a physical stat, a fear level, and stamina. Players may gain or lose stats based on game events. When the fear level reaches a certain threshold the player will freeze and become inactive.

Each player also has a behaviour type — **Rational**, **Aggressive**, **Cautious**, or **Unstable** — that determines how they respond to fear, game type, and decision-making. For example, Cautious characters avoid dangerous games, and Aggressive characters ignore fear but consume stamina.

7.2 Trust Relations

The system tracks trust-based relations between players. Trust depreciates over time without being solidified by mutual success or card trades. Betrayals invoke fear and alter the behaviour of the affected characters.

7.3 Player Behaviour Lifecycle (State Diagram)

A character starts in the **Rational** state. Transitions to other states occur based on the following triggers: fear spikes push Rational to **Cautious** and Cautious to **Unstable**; betrayal drives Rational or Cautious to **Aggressive**; prolonged aggression with low stamina turns Aggressive to Unstable; rest or trust-building can move Aggressive or Cautious back toward Rational, and Unstable to Cautious and then to Rational.

7.4 Games

Each game has a name, a type (mental or physical), and a difficulty level. Mental games require the mental stat, and physical ones the physical stat. The game can be played by a group of players; everyone whose appropriate stat is above the difficulty level wins. Players who win become less afraid, and players who lose become more afraid.

7.5 Cards

Players are rewarded with cards for winning a game. All cards possess a suit and number (e.g. 7 of Spades, 3 of Hearts). Having all 52 cards enables a player to exit the Borderland.

7.6 Arenas

Games occur in an arena. The arena has a name, a series of games, and an environmental hazard which affects players (e.g. fire, building collapse, mist). When the hazard activates, players become more afraid.

7.7 Game Master

The Game Master controls the games. They control the hazards in arenas and pick the next game randomly to create suspense.

7.8 Borderland Manager

The larger flow is controlled by the Borderland Manager. It sets up everything for the games, advances time by starting a game, and checks if the games are over. It can also observe the players and print them for the viewers.

The game ends when all the cards have been collected or when all the players are inactive. Play your survival, test yourself to limits, and try to get out of the Borderland!

7.9 Scenarios to model with sequence diagrams

7.9.1 Scenario 1: Typical Game

The Borderland Manager instructs the Game Master to begin the next round. The Game Master randomly selects a Game from the Arena's queue, notifies the Arena to set itself up, and then broadcasts a "game start" message to all Players. Each player responds by checking their relevant stat (mental or physical) against the game's difficulty and returning either "win" or "lose". The Game Master tallies results, updates each player's fear level, and awards cards to winners before reporting back to the Borderland Manager.

7.9.2 Scenario 2: Victory!

A Player wins their last needed card. The Game object notifies the Game Master of the player's victory. The Game Master updates the player's card collection, discovers the set now contains all 52 unique cards, and immediately informs the Borderland Manager. The Manager validates the full deck, broadcasts an "exit granted" message, orders the Arena to open an exit path, and logs the event for spectators while the victorious Player sends a final "leave borderland" confirmation back to both the Game Master and Manager.

Input file examples

players.txt

- Format

Name Mental Physical Behaviour

- Sample input

Maksud 7 5 Rational
Farizi 6 8 Aggressive
Silva 9 3 Cautious

games.txt

- Format

Name Type Difficulty

- Sample input

Bridge_Puzzle mental 6

Tag_Arena physical 5

Quiz_of_Death mental 7

arena.txt

- Format

Name Hazard

- Sample input

Abandoned_Theme_Park Fog

8 Wildlife Simulation

Wildlife conservation is a critical issue in a major game reserve. Trained rangers venture out daily to spot wildlife, administer medicine to diseased animals, and protect the habitat from poachers. The reserve is a well-organised operation where rangers, animals, habitats, and support vehicles coexist in symbiosis to foster nature's fragile balance.

8.1 Animals

All animals have a species, health, and a stress level. Animals become stressed or injured due to natural events or poachers. Rangers can calm them down and treat them.

Each animal also has a temperament profile — **Passive, Defensive, Curious, or Aggressive** — which influences the animal's reaction when approached. Aggressive animals attack or flee and stress surrounding animals. Defensive animals only defend themselves when cornered. Curious animals approach rangers uninvited. Temperament influences how rangers must prepare and interact.

8.2 Animal Behaviour Lifecycle (State Diagram)

Animals begin in a **Calm** state. As a ranger approaches, depending on their temperament, they may enter an **Alerted, Defensive, Curious, or Aggressive** state. While threatened or injured they may enter a **Fleeing or Attacking** state. Successfully administered treatment brings the animal to a **Stabilised** state, but severe injury or increasing stress can bring it to a **Critical** state that requires immediate attention.

8.3 Rangers

Rangers are diligent workers who protect the reserve's environment. Every ranger has a name, experience, efficiency, and a stress level. They can assist an animal by improving its status or reducing stress. They can detect animals that need help and help them. If they are in the same habitat as a poacher they find and fight the poacher; the winner is determined based on the randomly initialised strength of both the ranger and the poacher. Ranger stress builds up during dangerous rescues or poacher encounters; if stress becomes too great the ranger slows down or makes more dangerous choices.

8.4 Poachers

Poachers are intruders who hunt down animals. Each poacher targets a specific species. Different poachers are dangerous to a different degree. Poachers disturb wildlife and cause stress as they evade their hunters. They can also hurt the animals if they catch them.

8.5 Habitats

Each habitat has a name, a capacity, and the animals living in it. Animals can enter and leave habitats. If there is an incursion by a poacher it stresses all animals in the habitat.

8.6 Vehicles

Rangers travel in vehicles. Each vehicle has an id, fuel level, and a capacity. Vehicles can be refuelled and deployed to specific habitats. Vehicles also carry optional equipment — either tranquilliser kits or mobile medic units — depending on the type of vehicle, for use during emergencies.

8.7 Simulation

The simulation is run day by day. Each day, animals wander between habitats, rangers are sent to habitats to patrol them, and poachers go to habitats to hunt animals. The simulation reports at the end of the day the state of the reserve and the happenings on that day, for example, what animals were saved. Play it out, roll out your rangers, and save the wildlife!

8.8 Scenarios to model with sequence diagrams

8.8.1 Scenario 1: Hostile Animal Encounter

A ranger attempts to administer treatment to an injured animal with an Aggressive profile. The animal bites when the ranger gets close, so the ranger retreats and calls for help. Stress is increased, and the habitat is shut down as unsafe for a short period of time.

8.8.2 Scenario 2: Emergency Treatment in the Field

An attack by poachers leaves an animal critically injured. A ranger arrives in a vehicle equipped with a medic unit and speeds to the animal's rescue. The animal is stabilised on-site and is driven directly to the rehabilitation zone by the vehicle.

Input file examples

animals.txt

- Format

Species	Health	StressLevel	Temperament
---------	--------	-------------	-------------

- Sample input

Elephant	60	30	Defensive
Giraffe	70	20	Passive
Zebra	80	25	Curious

rangers.txt

- Format

Name	Experience	Efficiency
------	------------	------------

- Sample input

Balazs	5	7
Sofia	6	6

poachers.txt

- Format

Name TargetSpecies DangerLevel

- Sample input

Wilhelm Zebra 4
Anne Elephant 7

vehicles.txt

- Format

ID FuelLevel Capacity Equipment

- Sample input

V1 100 2 Tranquilliser_Kit
V2 100 3 Mobile_Medic_Unit

habitats.txt

- Format

Name Capacity

- Sample input

Savannah 3

9 Racing Tournament Simulation

During a high-speed race competition, professional drivers compete with diverse cars on challenging tracks. Every race sets new records of speed, agility, and endurance. Backstage, a system oversees cars, drivers, tracks, and the entire competition flow — from preparation to results.

9.1 Vehicles

Each vehicle has a model, speed, durability, and fuel. When a vehicle is driven it consumes fuel and its durability is reduced. Weather conditions also affect performance: rain decreases grip, affecting effective speed and fuel consumption, while fog decreases visibility, reducing speed even further and necessitating a pit stop.

Vehicles operate under a condition system with three levels:

- **Optimal** — the vehicle operates with all stats at full effectiveness.
- **Worn** — the vehicle experiences moderate speed loss.
- **Critical** — the vehicle misfires and requires an emergency fix. If durability drops too low the vehicle can become non-functional.

9.2 Vehicle Lifecycle (State Diagram)

At the start of a race the car is in the **Optimal** state. Depending on track wear and weather during the course of the race the car advances to the **Worn** state. The car depletes further until it reaches the **Critical** state where slowness and misfires occur. If a pit stop is performed successfully the car returns to Optimal; otherwise it may remain Critical and risk engine failure. Engine failure sets the car into a **Retired** state, ending the race for that car.

9.3 Drivers

Every driver has a name, skill, and a preferred vehicle model. The effective speed for a driver driving a vehicle is calculated based on driving skill and the maximum possible speed of the vehicle model. If a driver drives their preferred model their effective speed increases by 10%. Drivers pit stop to refuel or repair if needed in the course of a race.

9.4 Racetracks

A racetrack has a name, difficulty level, number of laps, and weather conditions. A given driver and car can compete on the track, racing lap by lap and keeping track of fuel, durability, speed, and weather effects. The difficulty level influences fuel consumption and wear and tear of the vehicle.

9.5 Mechanics

Mechanics offer the support needed to race. Each mechanic has a name and efficiency. Cars can be repaired or refuelled by mechanics. Mechanics are assigned to individual drivers on a race-by-race basis.

9.6 Race Events and Tournament

Race events are planned races with attending drivers and a track. After a race the results are reported and points are distributed to drivers according to their placements.

The race events are organised into a tournament where all drivers participate in each race. Each track is raced on exactly once. The results of the tournament are reported after each race and also at the end. The 1st, 2nd, and 3rd drivers are applauded on the podium at the end.

Race to victory!

9.7 Scenarios to model with sequence diagrams

9.7.1 Scenario 1: Driver Completing a Lap

Model the sequence of events when a driver completes a lap during a race. Show the interaction between the Driver, their Car, and the RaceTrack. Include the calculation of effective speed, fuel consumption, and durability reduction based on the car's condition state, driver's skill, preferred model bonus (if applicable), track difficulty, and current weather conditions.

9.7.2 Scenario 2: Pit Stop

Illustrate the process of a driver taking a pit stop to repair their car when its durability state becomes Critical. Show the Driver initiating the pit stop, interacting with the assigned Mechanic, the Mechanic performing repairs on the Car, and the Car's state potentially improving back to Optimal.

Input file examples

drivers.txt

- Format

Name Skill PreferredVehicle

- Sample input

Maksud 8 Ferrari
Farizi 6 BMW
Silva 7 BMW

vehicles.txt

- Format

Model Speed Durability Fuel

- Sample input

Ferrari 100 90 100
BMW 85 80 100
Porsche 95 85 100

mechanics.txt

- Format

Name Efficiency

- Sample input

Mike 5
Anna 7
Luca 6

tracks.txt

- Format

Name Difficulty Laps Weather

- Sample input

Desert_Drift 10 3 Clear
Urban_Dash 12 4 Rain
Mountain_Pass 15 3 Fog